

Numpy Basics

```
In [5]: import numpy as np
```

Basic array creation

```
In [8]: arr = np.array([2, 4, 5, 7, 8])
print(arr)
arr
```

```
[2 4 5 7 8]
```

```
Out[8]: array([2, 4, 5, 7, 8])
```

```
In [9]: listt = [1, 3, "234", "f"]
print(listt)
listt
```

```
[1, 3, '234', 'f']
```

```
Out[9]: [1, 3, '234', 'f']
```

```
In [10]: np.array(listt)
```

```
Out[10]: array(['1', '3', '234', 'f'], dtype='<U21')
```

Array Generation Function

```
In [13]: # arange
arr = np.arange(1, 10, 1)
```

```
In [14]: zeros = np.zeros(6)
zeros
```

```
Out[14]: array([0., 0., 0., 0., 0., 0.])
```

```
In [16]: zeros = np.zeros((6, 4))
zeros
```

```
Out[16]: array([[0., 0., 0., 0.],
                [0., 0., 0., 0.],
                [0., 0., 0., 0.],
                [0., 0., 0., 0.],
                [0., 0., 0., 0.],
                [0., 0., 0., 0.]])
```

```
In [17]: ones = np.ones((3, 3))
ones
```

```
Out[17]: array([[1., 1., 1.],
                [1., 1., 1.],
                [1., 1., 1.]])
```

```
In [19]: arr = np.linspace(1, 5, 1) #Return evenly spaced numbers over a specified interval.
arr
```

```
Out[19]: array([1.])
```

```
In [21]: arr = np.linspace(1, 5, 3)
arr
```

```
Out[21]: array([1., 3., 5.])
```

```
In [22]: arr = np.linspace(1, 5, 5)
arr
```

```
Out[22]: array([1., 2., 3., 4., 5.])
```

```
In [23]: arr = np.linspace(1, 5, 8)
arr
```

```
Out[23]: array([1.          , 1.57142857, 2.14285714, 2.71428571, 3.28571429,
                3.85714286, 4.42857143, 5.          ])
```

Random Generation Function

```
In [26]: a = np.random.rand(10)
a
```

```
Out[26]: array([0.77027176, 0.08588855, 0.39935781, 0.92633849, 0.08529707,  
              0.34670556, 0.39624199, 0.75455289, 0.04896225, 0.98911057])
```

```
In [29]: b = np.random.rand(0)  
b
```

```
Out[29]: array([], dtype=float64)
```

```
In [30]: np.random.randint(10)
```

```
Out[30]: 1
```

```
In [31]: np.random.randint(10, 20, 5)
```

```
Out[31]: array([10, 10, 16, 17, 17], dtype=int32)
```

Array Attributes

```
In [32]: arr = np.arange(1, 10).reshape(3, 3)  
arr
```

```
Out[32]: array([[1, 2, 3],  
              [4, 5, 6],  
              [7, 8, 9]])
```

```
In [33]: arr.shape
```

```
Out[33]: (3, 3)
```

```
In [34]: arr.size
```

```
Out[34]: 9
```

```
In [35]: arr.dtype
```

```
Out[35]: dtype('int64')
```

Array Methods

```
In [36]: arr
```

```
Out[36]: array([[1, 2, 3],  
              [4, 5, 6],  
              [7, 8, 9]])
```

```
In [38]: print(arr.min())
```

```
1
```

```
In [39]: print(arr.max())
```

```
9
```

```
In [40]: print(arr.sum())
```

```
45
```

```
In [41]: np.sum(arr, axis=1)
```

```
Out[41]: array([ 6, 15, 24])
```

```
In [42]: np.sum(arr, axis=0)
```

```
Out[42]: array([12, 15, 18])
```

```
In [43]: np.sum(arr)
```

```
Out[43]: np.int64(45)
```

```
In [44]: arr.mean()
```

```
Out[44]: np.float64(5.0)
```

```
In [45]: arr.std()
```

```
Out[45]: np.float64(2.581988897471611)
```

```
In [46]: arr.argmax()
```

```
Out[46]: np.int64(8)
```

```
In [47]: arr.argmax()
```

```
Out[47]: np.int64(0)
```

Reshaping and resizing

```
In [49]: arr = np.arange(1, 31)
arr
```

```
Out[49]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30])
```

```
In [50]: arr = arr.reshape(5, 6)
```

```
In [51]: arr
```

```
Out[51]: array([[ 1,  2,  3,  4,  5,  6],
 [ 7,  8,  9, 10, 11, 12],
 [13, 14, 15, 16, 17, 18],
 [19, 20, 21, 22, 23, 24],
 [25, 26, 27, 28, 29, 30]])
```

Numpy Indexing And Slicing Of Vectors

```
In [52]: arr = np.arange(11, 21)
arr
```

```
Out[52]: array([11, 12, 13, 14, 15, 16, 17, 18, 19, 20])
```

```
In [54]: arr_slice = arr[3:]
arr_slice
```

```
Out[54]: array([14, 15, 16, 17, 18, 19, 20])
```

```
In [55]: arr_slice = arr[:6]
arr_slice
```

```
Out[55]: array([11, 12, 13, 14, 15, 16])
```

```
In [56]: arr_slice = arr[3::2]
arr_slice
```

```
Out[56]: array([14, 16, 18, 20])
```

```
In [57]: arr[4]
```

```
Out[57]: np.int64(15)
```

NUMPY INDEXING AND SLICING OF MATRIX

```
In [59]: arr = np.arange(1, 31).reshape(5, 6)
```

```
In [60]: arr
```

```
Out[60]: array([[ 1,  2,  3,  4,  5,  6],
 [ 7,  8,  9, 10, 11, 12],
 [13, 14, 15, 16, 17, 18],
 [19, 20, 21, 22, 23, 24],
 [25, 26, 27, 28, 29, 30]])
```

```
In [61]: slice = arr[0:2, 1:3]
slice
```

```
Out[61]: array([[2, 3],
 [8, 9]])
```

```
In [62]: slice = arr[2:, 3:5]
slice
```

```
Out[62]: array([[16, 17],
 [22, 23],
 [28, 29]])
```

```
In [64]: slice = arr[0]
slice
```

```
Out[64]: array([1, 2, 3, 4, 5, 6])
```

```
In [65]: slice = arr[3]
slice
```

```
Out[65]: array([19, 20, 21, 22, 23, 24])
```

```
In [67]: slice = arr[:, 2]
         slice
```

```
Out[67]: array([ 3,  9, 15, 21, 27])
```

Boolean Indexing

```
In [68]: arr = np.arange(11, 21)
```

```
In [69]: arr
```

```
Out[69]: array([11, 12, 13, 14, 15, 16, 17, 18, 19, 20])
```

```
In [70]: bool_index = arr % 2 == 0
         bool_index
```

```
Out[70]: array([False,  True, False,  True, False,  True, False,  True, False,
                True])
```

```
In [71]: arr = arr[bool_index]
```

```
In [72]: arr
```

```
Out[72]: array([12, 14, 16, 18, 20])
```